



Predict the gridlock. Plan the response.

An AI command center that forecasts event-driven traffic across Bengaluru — and turns it into a ready-to-execute deployment plan, *before* the event begins.

Flipkart GRiDLOCK Hackathon 2.0 · Round 2

Theme · Event-Driven Congestion

THE PROBLEM

Bengaluru events break the city — and we plan for them **reactively**.

Cricket at Chinnaswamy, concerts at Palace Grounds, rallies at Freedom Park, marathons, sudden protests — each one chokes carriageways and junctions for hours.

- ◆ **Impact isn't quantified in advance.** Officers don't know which junctions will fail, or when.
- ◆ **Resource deployment is experience-driven.** Manpower, barricades and diversions are decided by gut, not data.
- ◆ **No post-event learning system.** The same mistakes repeat — nothing is measured against what actually happened.

THE SOLUTION

Forecast → Recommend → Learn

One workflow, mapped 1:1 to the problem statement.

01 · FORECAST

See it before it happens

ML predicts per-junction congestion over the full event timeline — arrival, peak, dispersal — isolating the event's own impact from baseline traffic.

02 · RECOMMEND

Turn it into a plan

Optimal manpower allocation, barricade points, and diversion routes — rendered live on the map, sized to a real staffing budget.

03 · LEARN

Get sharper each time

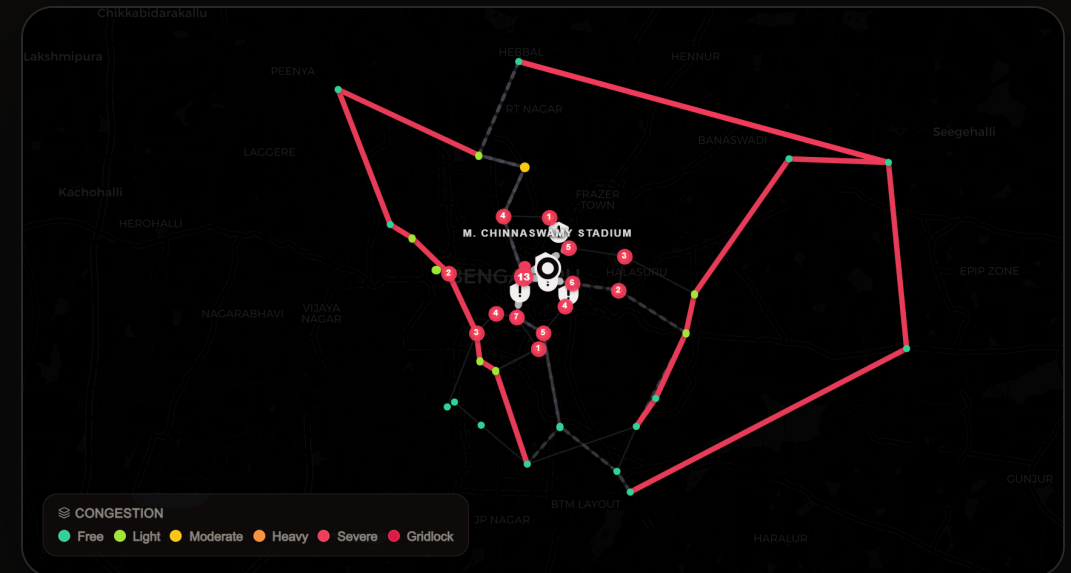
Predicted-vs-actual replay of past events with accuracy metrics — the post-event learning loop, built in.

A single command center

Conflux · scenario controls · live map · deployment plan

Congestion in space and time

- ◆ **38 real junctions** across central + arterial Bengaluru
- ◆ **Full timeline** arrival surge → peak → dispersal spike
- ◆ **Event-attributable impact** forecast minus normal baseline traffic
- ◆ **Delay, impact radius, worst junctions** as headline KPIs



A model that actually **learns**

A real road graph + a physics-based generator produce the "reality." The model sees only raw, knowable-in-advance features (attendance, type, distance, time, weather) and must learn the mapping — evaluated on **held-out, unseen events** (split by event, no leakage).

0.94

Congestion R^2 · MAE 3.0 / 100

0.82

Delay R^2 · MAE 0.49 min

79

Unseen events validated (~103k samples)

Drop-in ready: the same pipeline accepts real **ANPR / CCTV / loop-detector** feeds in production.

From forecast to field plan

MANPOWER

Where the officers go

Greedy marginal-utility allocation against a staffing budget — prioritising event-surge $\times$ congestion $\times$ network centrality, with expected delay reduction per junction.

BARRICADES

Control the inflow

Inflow-restriction points on the worst corridors near the venue, scored by impact and proximity.

DIVERSIONS

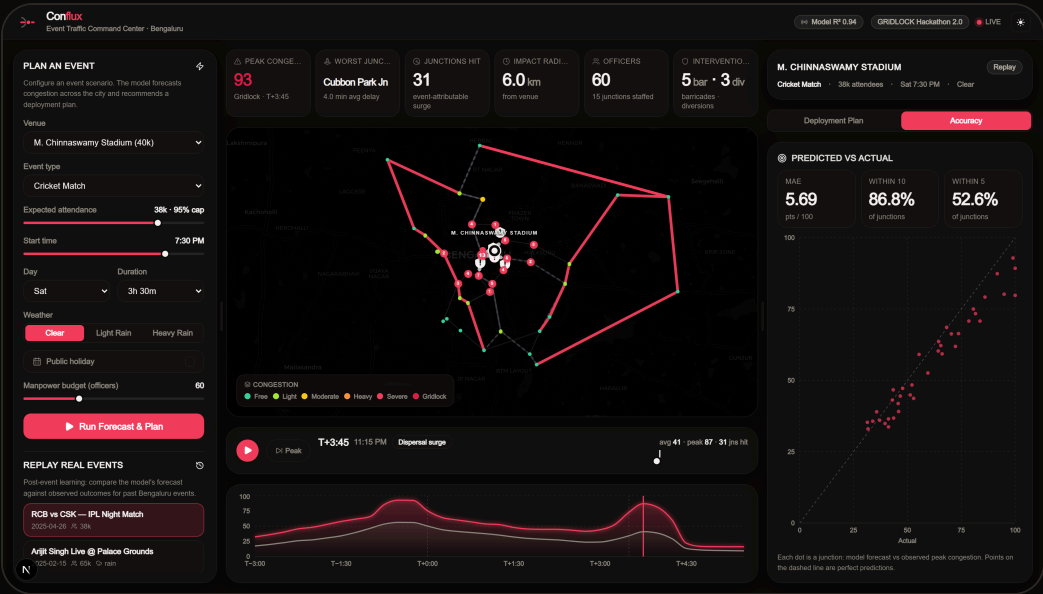
Keep the city moving

Congestion-aware re-routing of cross-city through-traffic around the event zone — computed on the live road graph.

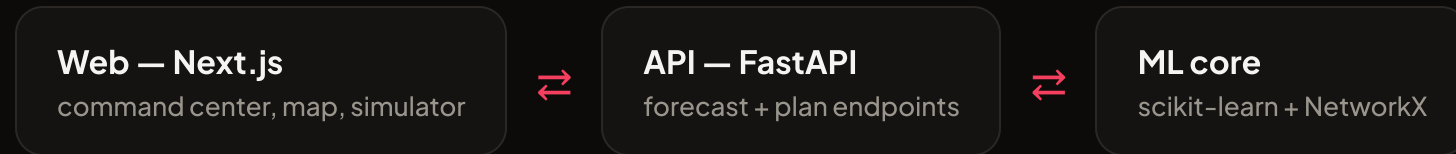
Predicted vs. actual

Replay any past event; every junction is a dot — forecast against observed peak. Points on the diagonal are perfect.

Event	MAE	Within 10
RCB vs CSK — IPL	5.7	87%
Arijit Singh — Palace Grounds	4.2	90%
Farmers' Rally — Freedom Park	3.0	100%
BFC Derby — Kanteerava	2.9	100%



Built to plug into the real city



- ◆ **Generalises to any venue × event type** graph + tabular model, no per-event tuning
- ◆ **Swap the data source, keep the pipeline** synthetic generator → live ANPR/CCTV/loop feeds
- ◆ **Deploys anywhere** web on Vercel, API as a container



Reactive chaos → **a plan, before the first car arrives.**

How many officers, where — what to barricade — what to divert. Quantified ahead of time, and sharper after every event.

Forecast

Recommend

Learn

Live demo conflux-chi-orpin.vercel.app

Code github.com/Komalpreet2809/conflux